

Guia Completo

JavaScript Moderno + Supabase

.map() · CRUD · Auth · Tempo Real

JavaScript ES6+

Supabase BaaS

React Ready

Full Stack

Este guia une o poder do método **.map()** com a praticidade do **Supabase**, mostrando desde os fundamentos até padrões avançados usados em produção. Aprenda a transformar dados, integrar backends e construir aplicações modernas com JavaScript.

1.1

O Conceito Fundamental

O método **.map()** é uma das ferramentas mais poderosas da programação funcional no JavaScript. Ele permite iterar sobre um array e criar um **novo array** com base nas transformações aplicadas — sem jamais alterar o array original.

javascript

```
const numeros = [1, 2, 3, 4];

// Cria novo array com valores duplicados
const duplicados = numeros.map(n => n * 2);

console.log(duplicados); // [2, 4, 6, 8]
console.log(numeros); // [1, 2, 3, 4] ← original intacto
```

■ **Imutabilidade:** .map() nunca modifica o array original. Cada chamada retorna um array completamente novo, o que torna o código mais previsível e fácil de depurar.

1.2

Anatomia da Função de Callback

A função passada ao **.map()** pode receber até três argumentos:

- elemento — o item atual do array.
- índice — a posição do item (0, 1, 2 ...).
- array — o array original completo.

javascript

```
const frutas = ['maçã', 'banana', 'laranja'];

const resultado = frutas.map((elemento, indice, arr) => {
  return `[${indice}] ${elemento} (total: ${arr.length})`;
});

// ['[0] maçã (total: 3)',
// '[1] banana (total: 3)',
// '[2] laranja (total: 3)']
```

1.3

Casos de Uso Comuns

Extraindo propriedades de objetos (JSON de API)

javascript

```
const produtos = [
  { id: 101, nome: 'Laptop', preco: 4500 },
  { id: 102, nome: 'Mouse', preco: 150 },
];

const soNomes = produtos.map(p => p.nome);
// ['Laptop', 'Mouse']

const comDesconto = produtos.map(p => ({
  ...p,
  preco: p.preco * 0.9, // 10% de desconto
}));
```

Formatando strings

javascript

```
const emails = ['Ana', 'Bruno', 'Carla'];

const slugs = emails.map(nome =>
  nome.toLowerCase().replace(/\s+/g, '-')
);
// ['ana', 'bruno', 'carla']
```

Encadeamento com .filter() e .reduce()

javascript

```
const pedidos = [
  { id: 1, valor: 200, pago: true },
  { id: 2, valor: 450, pago: false },
  { id: 3, valor: 100, pago: true },
];

const totalPago = pedidos
  .filter(p => p.pago) // só pedidos pagos
  .map(p => p.valor) // extrai valores
  .reduce((acc, v) => acc + v, 0); // soma

console.log(totalPago); // 300
```

1.4

Diferença Crucial: .map() vs .forEach()

Embora parecidos, os dois métodos têm propósitos distintos:

Método	Retorno	Quando usar
.map()	Novo Array	Transformar / converter dados
.forEach()	undefined	Executar efeitos colaterais (log, salvar no DB, chamar API)
.filter()	Novo Array (subset)	Filtrar itens por condição
.reduce()	Valor único	Agregar / acumular valores

■ ■ *Nunca use .map() apenas para efeitos colaterais (como console.log). Se não precisa do array retornado, prefira .forEach().*

1.5

.map() no React — Renderização de Listas

No React, o **.map()** é o padrão para renderizar listas de componentes dinamicamente a partir de dados. Cada item deve ter uma **prop key** única para que o React identifique mudanças de forma eficiente.

jsx

```
function ListaProdutos({ produtos }) {  
  return (  
  
    {produtos.map(produto => (  
  
      {produto.nome}  
      – R$ {produto.preco.toFixed(2)}  
  
    ))}  
  
  );  
}
```

■ Sempre use um identificador único (como id do banco) como key, nunca o índice do array — isso evita bugs de re-renderização quando a lista muda.

O **Supabase** é uma plataforma Backend-as-a-Service (BaaS) open-source, alternativa ao Firebase. Oferece banco de dados PostgreSQL, autenticação, armazenamento de ficheiros, funções serverless e funcionalidades em tempo real — tudo acessível via SDK JavaScript.

2.1

Instalação e Configuração

```
bash
```

```
npm install @supabase/supabase-js
```

Obtenha a **URL do projeto** e a **Chave API (anon key)** em *Settings > API* no dashboard do Supabase:

```
javascript
```

```
import { createClient } from '@supabase/supabase-js'

const supabaseUrl = 'https://seu-projeto.supabase.co'
const supabaseKey = 'sua-chave-anon-aqui'

export const supabase = createClient(supabaseUrl, supabaseKey)
```

■ ■ Nunca exponha sua `service_role` key no frontend! A anon key é segura para uso no cliente, pois o acesso é controlado pelas políticas RLS do banco.

2.2

Operações de Banco de Dados (CRUD)

CREATE — Inserir dados

```
javascript
```

```
const { data, error } = await supabase
  .from('usuarios')
  .insert([ { nome: 'Ana Silva', email: 'ana@email.com' } ])
  .select() // retorna o registro inserido

if (error) console.error(error);
else console.log('Inserido:', data);
```

READ — Consultar dados

javascript

```
// Buscar todos
const { data } = await supabase.from('usuarios').select('*')

// Selecionar colunas específicas
const { data } = await supabase
  .from('usuarios')
  .select('nome, email')

// Filtros encadeados
const { data } = await supabase
  .from('produtos')
  .select('*')
  .eq('categoria', 'eletronicos')
  .gte('preco', 100) // preco >= 100
  .order('preco', { ascending: true })
  .limit(10)
```

UPDATE — Atualizar dados

javascript

```
const { data, error } = await supabase
  .from('usuarios')
  .update({ nome: 'Ana Costa' })
  .eq('id', 1)
  .select()
```

DELETE — Remover dados

javascript

```
const { error } = await supabase
  .from('usuarios')
  .delete()
  .eq('id', 1)
```

2.3

Autenticação

javascript

```
// Cadastro de novo usuário
const { data, error } = await supabase.auth.signUp({
  email: 'exemplo@email.com',
  password: 'senha-segura',
})

// Login
const { data, error } = await supabase.auth.signInWithPassword({
  email: 'exemplo@email.com',
  password: 'senha-segura',
})

// Logout
await supabase.auth.signOut()

// Sessão atual
const { data: { user } } = await supabase.auth.getUser()
```

■ O Supabase também suporta login social (Google, GitHub, etc.) via OAuth — basta habilitar no dashboard em Authentication > Providers.

2.4

Row Level Security (RLS)

O RLS é o sistema de segurança do Supabase que controla quais linhas cada usuário pode ler ou modificar, diretamente no banco de dados.

sql

```
-- Habilitar RLS na tabela
ALTER TABLE posts ENABLE ROW LEVEL SECURITY;

-- Política: usuário só vê seus próprios posts
CREATE POLICY 'user_posts' ON posts
FOR SELECT USING (auth.uid() = user_id);

-- Política: usuário só insere posts para si mesmo
CREATE POLICY 'insert_own' ON posts
FOR INSERT WITH CHECK (auth.uid() = user_id);
```

■ ■ Se suas consultas retornam listas vazias mesmo com dados na tabela, verifique as políticas RLS no painel do Supabase.

2.5

Tempo Real (Realtime)

O Supabase permite escutar mudanças no banco em tempo real via WebSocket, ideal para chats, dashboards ao vivo e notificações.

javascript

```
const channel = supabase
  .channel('mudancas-posts')
  .on(
    'postgres_changes',
    { event: '*', schema: 'public', table: 'posts' },
    (payload) => {
      console.log('Mudança recebida:', payload)
    }
  )
  .subscribe()

// Cancelar escuta quando componente desmonta
// supabase.removeChannel(channel)
```


3.1

Buscando e Transformando Dados da API

O padrão mais comum no desenvolvimento moderno: buscar dados do Supabase e transformá-los com `.map()` antes de exibir.

javascript

```
async function getProdutosFormatados() {
  const { data, error } = await supabase
    .from('produtos')
    .select('id, nome, preco, estoque')
    .eq('ativo', true)

  if (error) throw error;

  // .map() transforma cada objeto para o formato da UI
  return data.map(produto => ({
    id: produto.id,
    label: produto.nome.toUpperCase(),
    precoLabel: `R$ ${produto.preco.toFixed(2)}`,
    disponivel: produto.estoque > 0,
    badge: produto.estoque < 5 ? '■ Últimas unidades' : null,
  }));
}
```

3.2

Componente React Completo

Exemplo de componente que combina Supabase para buscar dados e `.map()` para renderizar a lista:

jsx

```

import { useEffect, useState } from 'react'
import { supabase } from './supabaseClient'

export function ListaProdutos() {
  const [produtos, setProdutos] = useState([]);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    async function carregar() {
      const { data } = await supabase
        .from('produtos')
        .select('*')
        .order('nome');

      setProdutos(data ?? []);
      setLoading(false);
    }
    carregar();
  }, []);

  if (loading) return Carregando...;

  return (
    {produtos.map(p => (
      {p.nome} — R$ {p.preco.toFixed(2)}
    ))}
  );
}

```

3.3

Referência Rápida — Métodos Supabase

Método	Operação SQL	Exemplo
.select('*')	SELECT *	supabase.from('t').select('*')
.insert([...])	INSERT INTO	.insert([{{col: val}}])
.update({...})	UPDATE SET	.update({col: val}).eq('id', 1)
.delete()	DELETE FROM	.delete().eq('id', 1)
.eq(col, val)	WHERE col = val	.eq('status', 'ativo')

<code>.gte(col, val)</code>	WHERE col >= val	<code>.gte('preco', 100)</code>
<code>.ilike(col, '%x%')</code>	ILIKE (case insensitive)	<code>.ilike('nome', '%ana%')</code>
<code>.order(col)</code>	ORDER BY	<code>.order('criado_em', {ascending: false})</code>
<code>.limit(n)</code>	LIMIT n	<code>.limit(20)</code>

3.4

Boas Práticas — Resumo Final

Imutabilidade com <code>.map()</code>	Nunca modifique o array original. Use sempre o novo array retornado.
Tratamento de erros no Supabase	Sempre verifique o campo <code>error</code> antes de usar <code>data</code> nas queries.
Key no React	Use sempre o id único do banco como <code>key</code> ao renderizar listas com <code>.map()</code> .
RLS ligado	Ative Row Level Security em todas as tabelas que contêm dados de usuários.
Variáveis de ambiente	Guarde a URL e a anon key em variáveis de ambiente (<code>.env</code>), nunca no código.
<code>.map()</code> encadeado	Combine <code>.filter().map()</code> para filtrar e transformar em uma única pipeline.